

Universidad Rey Juan Carlos

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

DOBLE GRADO EN INGENIERÍA INFORMÁTICA E INGENIERÍA DE COMPUTADORES

CURSO ACADEMICO 2025-2026

TRABAJO FIN DE GRADO INGENIERÍA DE COMPUTADORES

**TELEGRAM COMO FUENTE DE INTELIGENCIA EN
CIBERSEGURIDAD: PIPELINE DE DETECCIÓN DE PHISHING
Y MENSAJES SOSPECHOSOS PARA ENTORNOS
EMPRESARIALES**

Autor: Alvaro Osuna Flores

Tutora: Liliana Patricia Santacruz Valencia

Resumen.....	1
1. Introduccion.....	2
1.1 Motivacion.....	2
1.2 Objetivos.....	3
1.3 Estructura del documento.....	4
2. Contexto.....	4
2.1 Telegram como superficie de interés en ciberseguridad.....	4
2.2 Necesidad de un enfoque de sistemas.....	5
2.3 Soluciones relacionadas y hueco del proyecto.....	5
2.4 Criterio de seguridad mínima.....	6
3. Metodología, requisitos y arquitectura.....	6
3.1 Metodología de trabajo.....	7
3.2 Requisitos funcionales.....	7
3.3 Requisitos no funcionales.....	8
3.4 Arquitectura general del pipeline.....	8
3.5 Flujo operativo y trazabilidad.....	9
3.6 Herramientas y tecnologías.....	10
3.7 Infraestructura y despliegue.....	11
4. Desarrollo e integración del sistema.....	12
4.1 Ingesta y gestión de sesión con Telegram.....	12
4.2 Preprocesado e inferencia.....	13
4.3 Persistencia y minimización de datos.....	13
4.4 Versionado de artefactos por ejecución.....	15
4.5 API y capa de consulta.....	15

4.6 Dashboard React y Grafana.....	16
4.7 Validacion integrada.....	17
5. Evaluacion y analisis de resultados.....	18
5.1 Metricas principales.....	18
5.2 Interpretación del umbral operativo.....	19
5.3 Matriz de confusión y distribución.....	19
5.4 Evidencia de validación integrada.....	20
5.5 Limitaciones detectadas.....	20
6. Conclusiones y líneas futuras.....	21
Bibliografía.....	22

Resumen

El presente trabajo presenta el diseño e integración de un sistema desacoplado para detectar mensajes de phishing y comunicaciones sospechosas procedentes de Telegram. El proyecto se enfoca desde el punto de la Ingeniería de Computadores como un pipeline operativo completo, en el que la captura, la cola de mensajes, la inferencia, la persistencia, la API y la observabilidad forman una misma cadena técnica.

La solución combina la librería Telethon para la escucha de mensajes, RabbitMQ para desacoplar la ingesta, un worker de inferencia basado en DistilBERT, la base de datos MongoDB para la persistencia trazable, FastAPI como contrato de consulta, un dashboard React para revisión funcional y Prometheus con Grafana para observabilidad en tiempo real.

Los artefactos de evaluación se versionan por el `run_id` y los benchmarks mediante el `benchmark_id`, lo que permite relacionar las métricas, las predicciones, la matriz de confusión, las evidencias de validación y las pruebas de carga por ejecución completa. En la evaluación offline de referencia se obtuvo `accuracy = 0.63`, `precision_pos = 0.5823`, `recall_pos = 0.92`, `f1_pos = 0.7132` y, en el benchmark controlado más exigente, el sistema mantuvo 19.08 mensajes/s reales con una latencia media E2E de 52.4 ms y p95 de 66.06 ms.

El punto más valioso del proyecto, visto desde la Ingeniería de Computadores; es que cada fase del ciclo de vida del mensaje queda conectada con la siguiente y, además, queda también medida. La entrada en Telegram, la publicación en cola, el consumo por el worker, la escritura en MongoDB, la exposición por API y la monitorización en métricas y dashboards (Prometheus/Grafana) generan evidencia para auditar latencias, uso de recursos y capacidad del pipeline.

Aunque el entrenamiento del modelo no constituye el eje principal de esta memoria, tampoco se ha tratado la IA como un sistema opaco. El sistema se apoya en un modelo derivado de `distilbert-base-uncased` con metadatos de entrenamiento documentados, checkpoints conservados y una ruta de carga preparada para funcionar tanto con artefactos locales como con el repositorio versionado del proyecto.

Quiero destacar del trabajo no es solo el clasificador, sino la integración de productor, cola, worker, persistencia, API y observabilidad en un flujo trazable y defendible, orientado a un escenario realista de operación y pensado para absorber picos de carga y facilitar el mantenimiento.

1. Introduccion

1.1 Motivacion

Telegram se ha convertido en un entorno de interés para la ciberseguridad por dos motivos principales. Por un lado, es una plataforma ampliamente usada para comunicación, coordinación y difusión rápida de mensajes. Por otro lado, sus canales, grupos y dinámica de propagación privadas y cifradas de extremo a extremo facilitan también la distribución de enlaces fraudulentos, señuelos de phishing, credenciales comprometidas o conversaciones relevantes sobre el malware general (Europol, 2022; Guo et al., 2024).

Desde una perspectiva operacional, el problema no consiste únicamente en clasificar los mensajes de texto. El reto real es construir un sistema capaz de recibir mensajes en tiempo prácticamente real, tratarlos de manera consistente, decidir con qué evidencia se almacenan, exponer resultados a otros componentes y mantener observabilidad suficiente para una revisión posterior. Esa naturaleza integrada hace que el proyecto encaje especialmente bien en la visión de la Ingeniería de Computadores.

Además, un análisis manual continuo resulta costoso, muy pesado, poco escalable y propenso a errores. En consecuencia, es razonable automatizar la primera clasificación, siempre que el sistema deje trazabilidad técnica suficiente para justificar la salida producida y para explicar como se ha llegado a ella.

Esta forma de plantearlo ayuda también a diferenciar con claridad este trabajo del futuro TFG de fake news. Mientras aquel puede centrarse con mayor naturalidad en datos, preprocesado, entrenamiento, evaluación y lógica de aplicación, aquí el peso recae en la arquitectura del

pipeline, en la integración entre módulos y en las decisiones de ejecución, almacenamiento y observabilidad necesarias para operar el sistema.

1.2 Objetivos

Objetivo general

Desarrollar un pipeline modular para capturar mensajes de Telegram, publicarlos en una cola RabbitMQ, consumirlos desde un worker de inferencia y persistir evidencia técnica suficiente para consultar, monitorizar y evaluar el sistema de forma reproducible.

En esta memoria, ese objetivo se entiende además como el diseño de un sistema desplegable, auditable y medible. Por eso, además del resultado de la clasificación, se ha buscado justificar como se desacoplan los componentes, como se observan las latencias por etapa y cómo se valida el comportamiento con benchmark y pruebas.

Objetivos específicos

- Implementar la ingesta automática de mensajes desde Telegram y desacoplar mediante una cola RabbitMQ.
- Integrar un worker de preprocesado e inferencia con un modelo binario entrenado previamente.
- Persistir evidencia técnica suficiente en MongoDB, incluyendo latencias por etapa y snapshots de recursos, sin almacenar datos sensibles más de lo estrictamente necesario.
- Versionar los artefactos de evaluación y benchmark por un `run_id` y un `benchmark_id` para mantener coherencia entre ejecuciones, métricas y endpoints.
- Exponer resultados mediante una API reutilizable, métricas mediante Prometheus y visualizaciones en React y Grafana.
- Validar el sistema con pruebas de API, benchmark controlado y casos simulados de integración.
- Definir una topología de ejecución reproducible mediante contenedores y perfiles de despliegue, separando productor, cola, worker, API y observabilidad.
- Documentar la procedencia del modelo y las principales decisiones de arquitectura para justificar la integración de IA, la elección de RabbitMQ, MongoDB y Prometheus y el enfoque de sistemas adoptado.

1.3 Estructura del documento

La memoria se organiza en seis bloques. En primer lugar se presenta el contexto del problema y la justificación del enfoque adoptado. A continuación, se describen la metodología, los requisitos y la arquitectura desacoplada. Después, se detalla la integración del productor, la cola, el worker, la API y la observabilidad. Para que finalmente se analizan los resultados offline y del benchmark, y se cierran las conclusiones y líneas futuras.

2. Contexto

2.1 Telegram como superficie de interés en ciberseguridad

Los análisis recientes y los informes del sector coinciden en que Telegram se ha consolidado como una fuente dual: sirve tanto para comunicaciones legítimas como para actividades ilícitas y de coordinación criminal (Europol, 2022; Guo et al., 2024). Esa dualidad lo convierte en un espacio útil para la experimentación acerca de la inteligencia en ciberseguridad.

Para las empresas, esto tiene una consecuencia clara: conviene disponer de mecanismos que permitan observar, filtrar y priorizar mensajes potencialmente peligrosos sin depender exclusivamente de la revisión manual. De forma particular, el phishing sigue siendo una de las amenazas más frecuentes y más efectivas, por lo que detectar alertas tempranas en mensajes o enlaces compartidos puede reducir el tiempo de reacción ampliamente.

2.2 Necesidad de un enfoque de sistemas

En este trabajo no basta con responder si un mensaje parece benigno o peligroso. La pregunta relevante es más amplia:

- Cómo llega el mensaje al sistema
- Cómo se procesa y clasifica
- Qué datos se conservan
- Cómo se audita la ejecución
- Como se consumen los resultados desde otros componentes

Estas preguntas obligan a pensar el problema como un pipeline de sistemas. Por eso, el proyecto no se ha orientado solo a la mejora de un clasificador, sino al ensamblaje de varios módulos coordinados que permitan desplegar una solución, para validarla y operar.

Visto de esta manera, el mensaje deja de ser un simple texto y pasa a ser una unidad de trabajo que atraviesa varias capas de procesado. Primero hay una capa de adquisición, responsable de mantener la sesión con Telegram y recibir eventos de nuevos mensajes(NewMessage). Después, entra en juego una capa de transformación, donde el contenido se normaliza, se detecta el idioma y se prepara la entrada del modelo. A continuación actúa la capa de decisión, que calcula el score_1, aplica el umbral operativo y adjunta los metadatos del modelo y del dispositivo. En la última fase aparecen las capas de persistencia y explotación, las cuales almacenan el evento, lo publican mediante endpoints y lo convierten en una fuente de consulta operativa para poder ser usados por React y Grafana.

El interés académico no se limita a la calidad del clasificador, sino en la capacidad de integrar comunicaciones, almacenamiento, servicios, observabilidad y controles mínimos de seguridad en un sistema coherente.

2.3 Soluciones relacionadas y hueco del proyecto

Existen algunas soluciones comerciales y académicas para monitorizar fuentes abiertas, orquestar indicadores o aplicar modelos de lenguaje a ciberseguridad. Por contraparte, muchas de esas soluciones son generalistas, costosas o están poco centradas en una integración ligera y modular sobre sistemas concretos. El hueco que cubre este TFG consiste en demostrar una arquitectura defendible, con tecnologías accesibles y con especial atención en:

- Desacoplamiento ligero de la ingesta mediante RabbitMQ
- Almacenamiento operativo flexible en MongoDB
- Contrato de consulta y explotación mediante API;
- Observabilidad con Prometheus y Grafana;
- Trazabilidad por ejecución y benchmark.

2.4 Criterio de seguridad mínima

Tras revisiones de tutoría se optó por reforzar el sistema en un aspecto importante, la explotación mínima segura, es decir; el nivel mínimo exigible para operar sin comprometer la seguridad. . A partir de esta definición se fijaron cinco criterios:

- 1.No exponer MongoDB ni la cola RabbitMQ fuera del entorno local salvo puertos controlados de demo
- 2.Exigir autenticación simple en la API;
- 3.Desactivar acceso anónimo en Grafana;
- 4.Restringir CORS a orígenes concretos;
- 5.Minimizar los datos personales o textuales persistidos en MongoDB.

Este conjunto de decisiones no convierte el sistema en una plataforma final de producción, pero si lo hace más coherente y defendible desde el punto de vista de la seguridad informática. En particular, la consola de RabbitMQ queda como herramienta de demostración y no como superficie abierta por defecto.

3. Metodología, requisitos y arquitectura

3.1 Metodología de trabajo

El proyecto se ha desarrollado con una metodología incremental, adoptando muchos conceptos de las metodologías ágiles. Primero, se construyó un núcleo funcional de captura, inferencia y almacenamiento. Posteriormente, se reorganizó en torno a los artefactos versionados por `run_id`. Finalmente, se introdujeron RabbitMQ para desacoplar ingesta e inferencia, Prometheus para observabilidad real y un benchmark controlado para estudiar latencia, throughput y consumo de recursos.

3.2 Requisitos funcionales

Código	Requisito
RF-1	Capturar mensajes de Telegram y publicarlos en RabbitMQ mediante un productor desacoplado.

RF-2	Consumir la cola, preprocesar el texto y ejecutar inferencia binaria con un worker especializado.
RF-3	Guardar una traza técnica mínima en MongoDB, incluyendo latencias por etapa y recursos.
RF-4	Generar artefactos reproducibles de evaluación y benchmark por run_id y benchmark_id.
RF-5	Publicar resultados mediante API y métricas de Prometheus, y visualizarlos en React y Grafana.
RF-6	Validar el comportamiento con pruebas automatizadas, benchmark y casos simulados.

3.3 Requisitos no funcionales

Código	Requisito
RNF-1	La solución debe ser modular, desplegable localmente y compatible con ejecución por perfiles.
RNF-2	La ingesta y la inferencia deben poder desacoplarse para absorber picos temporales de carga.
RNF-3	Los resultados deben ser trazables por ejecución, benchmark y artefacto asociado.
RNF-4	La observabilidad debe incluir métricas de latencia, throughput y recursos del proceso.
RNF-5	La persistencia por defecto debe reducir la exposición de datos sensibles y mantener controles mínimos de acceso.

3.4 Arquitectura general del pipeline

La arquitectura se organiza en cinco bloques:

- Captura de mensajes y publicación en cola desde Telegram
- Consumo, preprocesado e inferencia en el worker
- Persistencia operativa y versionado de artefactos
- Publicación y consulta mediante API y dashboard React
- Observabilidad y benchmark con Prometheus, Grafana y scripts/logs .

Arquitectura funcional del sistema TFG

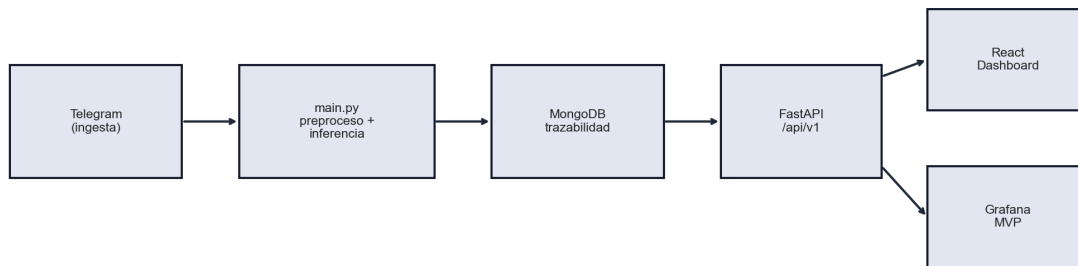


Figura 1. Arquitectura general del sistema con productor, RabbitMQ, worker, MongoDB, API, React y observabilidad.

La capa de entrada operativa ya no se limita a un script monolítico. El archivo `main.py` se mantiene como punto de entrada compatible con modos legacy, productor y worker, mientras que `producer.py` y `worker.py` separan explícitamente la captura desde Telegram de la inferencia y la persistencia.

En bajo nivel, en la arquitectura lógica a la infraestructura real del proyecto, estos bloques se corresponden con servicios diferenciados. El stack base levanta la base de datos, la cola, el worker, la api y el frontend. El perfil advanced añade el productor real de Telegram. El perfil observability incorpora Prometheus y Grafana. Esta topología permite desacoplar picos de entrada, medir el comportamiento por etapa y escalar consumidores sin reescribir el resto del pipeline.

3.5 Flujo operativo y trazabilidad

El flujo del mensaje recibido es el siguiente:

- Se entrega el mensaje recibido de Telegram al productor autenticado.
- El productor publica un evento en RabbitMQ con la información mínima necesaria para el procesamiento.

- El worker consume la cola, normaliza el texto, ejecuta la inferencia y decide la clase final según un umbral configurable.
- Se genera una traza con identificadores, el hash del mensaje, las latencias por etapa, los snapshots de recursos y los metadatos del modelo.
- La información queda disponible para persistencia, consulta por API, scraping de Prometheus y visualización operativa.

En la práctica, este recorrido se convierte en una cadena, por los scripts, de transformaciones muy concretas. El script `producer.py` valida las credenciales de Telegram y publica envelopes con timestamps de origen y cola. El script `worker.py` consume esos mensajes, llama al script `pipeline.py` para construir el documento técnico completo y, finalmente, persiste la evidencia en MongoDB.

Este comportamiento es importante porque muestra que la trazabilidad no se añade a posteriori. La evidencia técnica se genera en el mismo recorrido del mensaje y acompaña a la decisión del modelo con tiempos de preprocesado, inferencia, espera en cola, escritura en base de datos y uso de CPU y memoria del proceso.

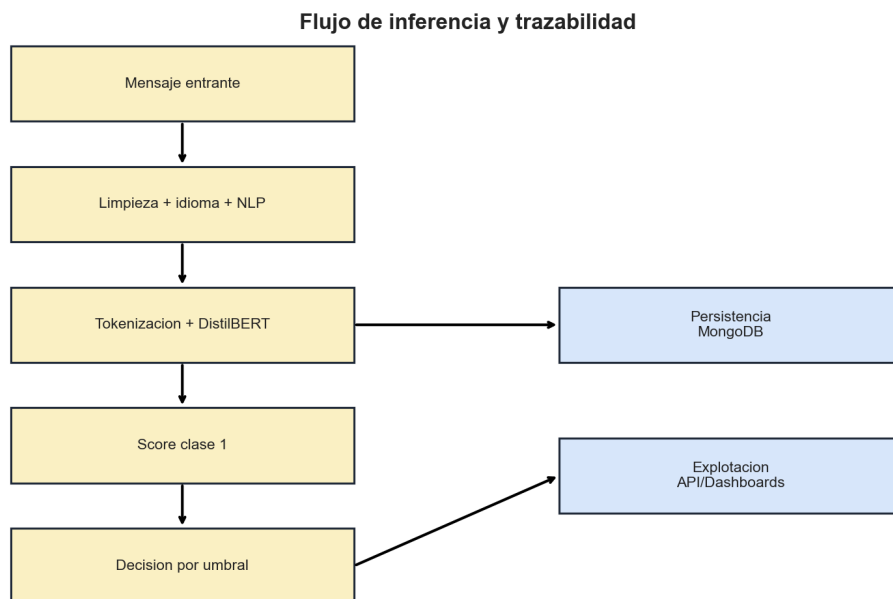


Figura 2. Flujo de productor, cola, worker y trazabilidad aplicado a cada mensaje procesado. Esta secuencia es precisamente la que refuerza el enfoque de sistemas del TFG. La salida del modelo no se trata como un dato aislado, sino como un evento que atraviesa distintas capas técnicas absorbiendo picos de carga mediante la cola y dejando mediciones suficientes para evaluar estabilidad y capacidad.

3.6 Herramientas y tecnologías

Categoría	Herramientas	Función dentro del sistema
Ingesta	Python, Telethon, RabbitMQ	Conexión a Telegram y publicación desacoplada de mensajes
IA	Hugging Face Transformers, PyTorch	Carga del modelo e inferencia binaria
Persistencia	MongoDB	Almacenamiento operativo de trazas y metadatos
Explotacion	FastAPI, Pydantic	Publicacion de resultados, estadísticas y contratos de consulta

Observabilidad	Prometheus, Grafana, psutil	Métricas temporales, paneles y snapshots de recursos
Visualización	React	Consulta funcional de runs, mensajes y benchmarks
Integración	Docker, docker compose, pytest, GitHub Actions	Despliegue local y validación automatizada

3.7 Infraestructura y despliegue

El programa está preparado para ejecutarse localmente mediante docker compose con perfiles diferenciados. El contenedor base incluye MongoDB, Rabbitmq, el worker y la api. El perfil observability incorpora Prometheus y Grafana. El perfil advanced activa el productor real sobre Telegram. Esta organización facilita la demostración, depuración y defensa del sistema sin obligar a levantar siempre todos los componentes.

El endurecimiento mínimo introducido en docker-compose.yml, añade una capa de seguridad, evita la exposición pública de MongoDB, deja la API protegida por X-API-Key, restringe CORS a orígenes concretos y obliga a mantener Grafana con autenticación. La consola de RabbitMQ se conserva porque resulta útil en la demo, pero se entiende como un recurso de laboratorio local, no como interfaz abierta de producción.

La topología definida en docker-compose.yml despliega siete servicios principales. MongoDB conserva los datos en un volumen dedicado, Rabbitmq desacopla productor y consumidor, el worker ejecuta la inferencia y expone métricas, la api sirve resultados y agregados, el frontend ofrece consulta funcional, Prometheus centraliza el scraping y Grafana presenta paneles de rendimiento y capacidad. El productor de Telegram permanece separado como servicio opcional. Esta separación aporta ventajas prácticas desde la perspectiva de sistemas. RabbitMQ se ha escogido frente a Kafka porque el patrón productor-consumidor del TFG es sencillo, la consola facilita explicar el desacoplamiento y la sobrecarga operativa es menor para un prototipo académico. MongoDB, por su parte, encaja bien como almacenamiento operativo porque permite persistir documentos heterogéneos con trazas y metadatos sin forzar un esquema rígido.

Conviene destacar que la observabilidad ya no se resuelve con un JSON servido a Grafana. Prometheus actúa como fuente real de métricas temporales, scrapeando tanto la API como el worker y permite construir paneles de throughput, latencia p95, profundidad de cola, CPU y

memoria. El benchmark controlado completa esa observabilidad con escenarios capaces de repetir 1, 5, 10 y 20 mensajes por segundo.

4. Desarrollo e integración del sistema

4.1 Ingesta y gestión de sesión con Telegram

El punto de entrada operativo se reparte ahora entre `producer.py`, `worker.py` y `main.py`. El productor válida `TELEGRAM_API_ID`, `TELEGRAM_API_HASH` y `TELEGRAM_PHONE`, crea la sesión de Telethon y escucha eventos `NewMessage` para transformar cada mensaje en un envelope ligero apto para publicarse en RabbitMQ.

Una vez iniciada la sesión, la ingesta queda desacoplada del procesamiento. Esta decisión, permite absorber picos temporales y separar claramente la responsabilidad de captura de la responsabilidad de inferencia. El archivo `main.py` conserva un modo legacy para no romper compatibilidad con el MVP anterior, pero la arquitectura defendida en esta memoria es la ruta `producer -> rabbitmq -> worker`.

4.2 Preprocesado e inferencia

El preprocesado actual se centraliza en el script `pipeline.py` e incluye normalización del texto, reducción de espacios, conversión a minúsculas, limpieza de caracteres y detección del idioma cuando la información está disponible.

La salida principal del modelo reside en `score_1`, interpretado como la probabilidad de amenaza. La decisión binaria final se calcula aplicando `THRESHOLD`. Para mantener compatibilidad con estados previos del proyecto se conserva `latency_ms`, pero ahora se acompaña de `preprocess_latency_ms`, `inference_latency_ms`, `db_write_latency_ms`, `queue_wait_latency_ms` y `end_to_end_latency_ms`.

Aunque esta memoria no desarrolla el entrenamiento con el detalle propio del TFG de Informática, si conviene explicar de dónde sale el modelo integrado. El archivo `model_loader.py` intenta reutilizar artefactos locales entrenados y, cuando existen, recupera la misma configuración documentada en `training_metadata.json` para mantener coherencia entre evaluación offline e inferencia operacional.

La evidencia disponible en docs/training_metadata.json, scripts_experimentos/, datos_entrenamiento/ y modelos_entrenados/ muestra además que el modelo no se ha seleccionado de forma casual. El componente de IA queda integrado como una pieza concreta del sistema, no como un servicio externo opaco.

4.3 Persistencia y minimización de datos

La colección MongoDB no guarda por defecto el texto original. La entrada sobre la base de datos incluye:

- run_id y source;
- created_at_utc, source_received_at_utc, queued_at_utc y persisted_at_utc;
- user_hash, chat_hash y message_id;
- msg_sha256, channel e idioma;
- pred, score_1 y threshold;
- preprocess_latency_ms, inference_latency_ms, db_write_latency_ms y end_to_end_latency_ms;
- queue_wait_latency_ms y queue_depth;
- cpu_percent, rss_bytes, vms_bytes y gpu_memory_bytes cuando aplica;
- ok, error, hf_model, model_source y device.

Esta estrategia ofrece un equilibrio útil entre trazabilidad y privacidad. El sistema conserva una huella del contenido, identifica la ejecución concreta y permite auditar la inferencia sin depender del texto original salvo configuración explícita. Además, las nuevas latencias por etapa y snapshots de recursos convierten cada documento en una evidencia técnica más completa.

La función ensure_indexes() crea índices por run_id, msg_sha256 y timestamps, y puede aplicar retención automática sobre created_at_utc para limitar el historico operativo si así se configura.

La pseudonimización se realiza mediante privacy_utils.py, que genera user_hash y chat_hash con sha256 y una sal configurable. Junto a ello, las variables STORE_MSG_ORIGINAL, STORE_CHANNEL_NAME y STORE_USER_FIELDS permiten graduar el nivel de detalle persistido y mantener el principio de minimización de datos.

4.4 Versionado de artefactos por ejecución

Una mejora estructural importante fue dejar de tratar la evaluación como un estado global único. A partir de las revisiones, el archivo `evaluate.py` crea artefactos en `reports/runs/<run_id>/` y el benchmark controlado hace lo mismo en `reports/benchmarks/<benchmark_id>/`. De esta forma, cada evidencia queda asociada a una ejecución concreta y deja de sobrescribirse al repetir experimentos.

Esta decisión afecta a toda la capa de explotación:

- La API puede servir la matriz de confusión o incluso el análisis por umbral correctos para un `run_id` concreto
- El dashboard React consulta datos coherentes con la ejecución seleccionada y lista benchmarks históricos
- Los scripts de validación y benchmark pueden comparar artefactos sin perder la evidencia previa.

La organización de artefactos en `reports/runs/<run_id>/` y `reports/benchmarks/<benchmark_id>/` tiene una clara ventaja arquitectónica, separa el histórico de la copia rápida del último estado. El script `reporting.py` mantiene los directorios versionados y, al mismo tiempo, deja espejos resumidos en la raíz de `reports/` para consumo operativo.

De cara a la integración, el `run_id` y el `benchmark_id` actúan como identificadores entre el clasificador offline, el benchmark, la API, el dashboard web y la memoria del proyecto. Gracias a ello, una consulta, un snapshot o un simple screenshot puede trazarse fácilmente hasta el artefacto que la respalda.

4.5 API y capa de consulta

La API FastAPI actúa como interfaz intermedia entre la persistencia, los artefactos versionados y las herramientas de explotación. Expone endpoints para health, ejecuciones, resumen de métricas, umbrales, matriz de confusión, mensajes persistidos, estadísticas, benchmarks y metadatos de entrenamiento. También publica `/metrics` como superficie de scraping para Prometheus.

La aplicación utiliza una construcción perezosa (LazyFastAPIApp) para evitar que la importación falle cuando aún no se han definido ciertas variables de entorno. Esta decisión mejora la robustez

del despliegue y facilita el arranque en contenedores y la ejecución de pruebas. Al mismo tiempo, el archivo `worker.py` expone sus propias métricas de proceso y de pipeline para completar la observabilidad.

La API debe entenderse como el contrato operativo del sistema, no solo como una lista de endpoints. El archivo `api/app.py` centraliza la autenticación mediante dependencia, validación de parámetros y serialización de respuestas. El archivo `api/services.py` agrega latencias, percentiles, throughput y consumo medio de recursos a partir de las trazas persistentes.

La propia configuración refuerza el enfoque de despliegue defendible. El archivo `api/settings.py` exige `API_KEY`, define orígenes CORS concretos y resuelve rutas hacia `reports/` y `docs/training_metadata.json`.

4.6 Dashboard React y Grafana

La capa web permite consultar rápidamente el estado de las ejecuciones, los mensajes que persisten y los benchmarks disponibles desde una interfaz simple y entendible. React se utiliza como frontend operativo de consulta, mientras que Grafana se reserva para el seguimiento temporal de la carga y la salud del pipeline sobre datos de Prometheus.



Figura 3. Vista resumida del dashboard React para exploración de runs, mensajes y benchmarks. En adición, el dashboard incorpora una pantalla de rendimiento que facilita revisar la evolución de las métricas principales y el contrastar con los benchmarks guardados. Esto resulta útil durante la

defensa porque permite relacionar una ejecución offline concreta con el comportamiento del sistema bajo carga controlada.

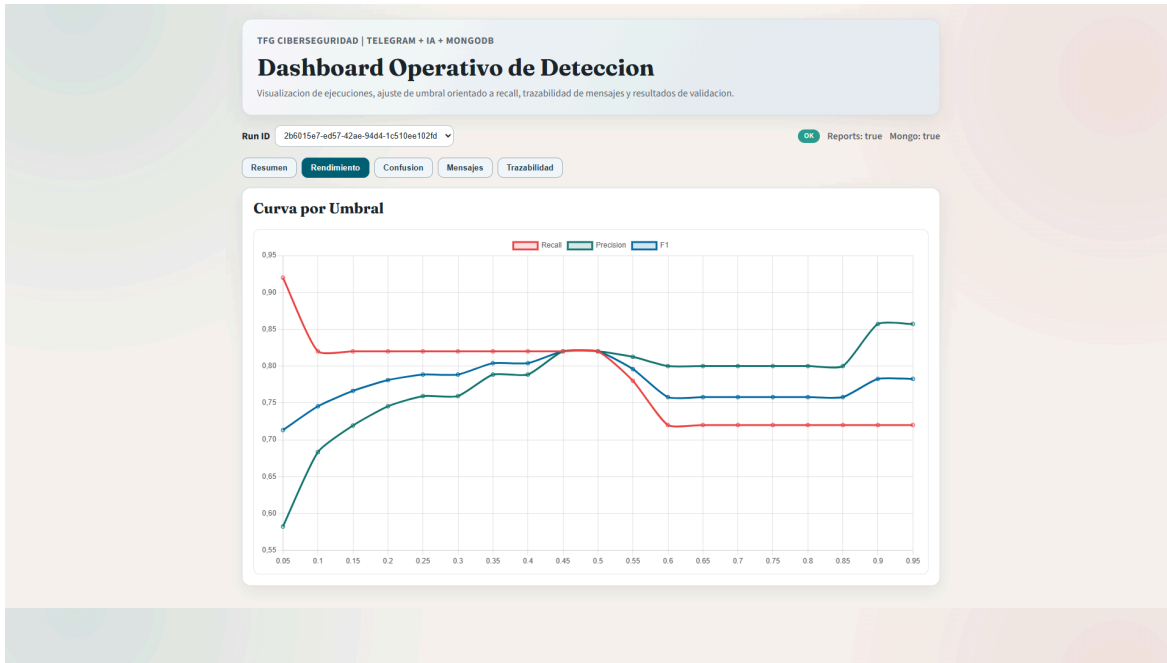


Figura 5. Vista del dashboard React orientada al seguimiento de latencias por etapa y benchmarks.

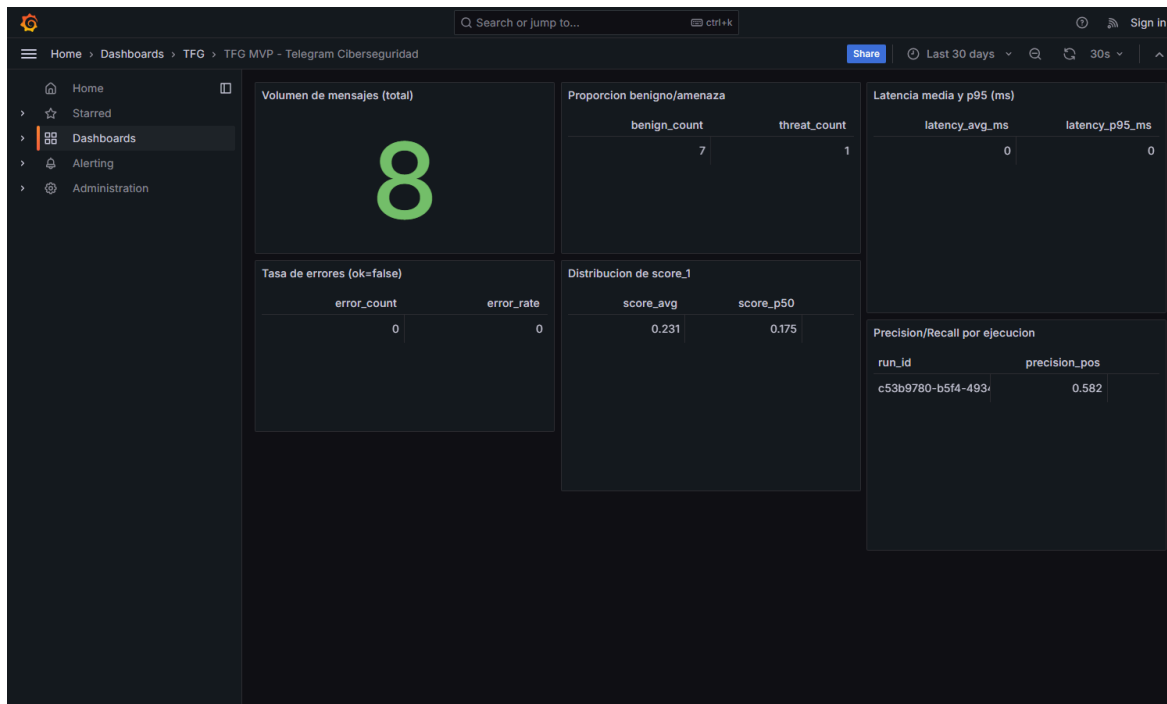


Figura 6. Panel de Grafana sobre Prometheus para monitorizacion operativa del pipeline.

La coexistencia de ambas capas refuerza la idea de pipeline explotable, la API sirve como contrato técnico común para consulta funcional y React se orienta a revisión por ejecución concreta. Prometheus y Grafana, en cambio, se especializan en series temporales, percentiles, profundidad de cola y consumo de recursos.

El dashboard React y Grafana no duplican exactamente el mismo papel. El primero permite inspeccionar ejecuciones concretas, revisar mensajes, filtrar por predicción o score y consultar benchmarks históricos. Grafana ofrece paneles vivos de throughput, latencia p95, CPU, memoria y cola, sin depender de lecturas manuales sobre MongoDB.

Este detalle de integración es relevante porque evita el acceso directo de Grafana a MongoDB. La API queda protegida para consulta funcional y Prometheus se consolida como la fuente especializada de observabilidad.

4.7 Validacion integrada

El proyecto incorpora varios niveles de verificación:

- Pruebas pytest para la API y la capa de servicios

- Evaluación offline reproducible por `run_id`
- Simulación de casos operativos y benchmark controlado con 1, 5, 10 y 20 mensajes por segundo
- Un runner de integración y comprobaciones básicas en GitHub Actions.

La observación nos ha dirigido a que la validación no se apoye únicamente en tests unitarios. El archivo `scripts/run_phase5_checks.py` comprueba artefactos clave, el archivo `benchmark_pipeline.py` genera evidencias cuantitativas y el workflow de GitHub Actions ejecuta una batería mínima reproducible sobre ambos TFGs.

5. Evaluacion y analisis de resultados

5.1 Metricas principales

La evaluación offline de referencia, almacenada en `reports/runs/63c1d8a6-613a-4971-b3bf-dc0fe2907001/metrics.json`, ofrece los siguientes resultados sobre 100 muestras balanceadas:

- `accuracy` = 0.63
- `tasa_error_prueba` = 0.37
- `precision_pos` = 0.5823
- `recall_pos` = 0.92
- `f1_pos` = 0.7132
- `roc_auc` = 0.8684
- `average_precision` = 0.8951

La tasa de error en datos de prueba se obtiene directamente como $1 - \text{accuracy}$, por lo que el sistema falla en 37 de cada 100 muestras del conjunto offline utilizado en la evaluación. Este dato obliga a interpretar el clasificador como una primera criba para posterior clasificación manual y no como una decisión final automática.

Estas métricas no deben leerse de forma aislada. En este trabajo interesa comprobar que el pipeline mantiene trazabilidad, estabilidad y tiempos de respuesta aceptables cuando se integra

como sistema. Por eso la evaluación offline se complementa con benchmark controlado y monitorización de recursos.

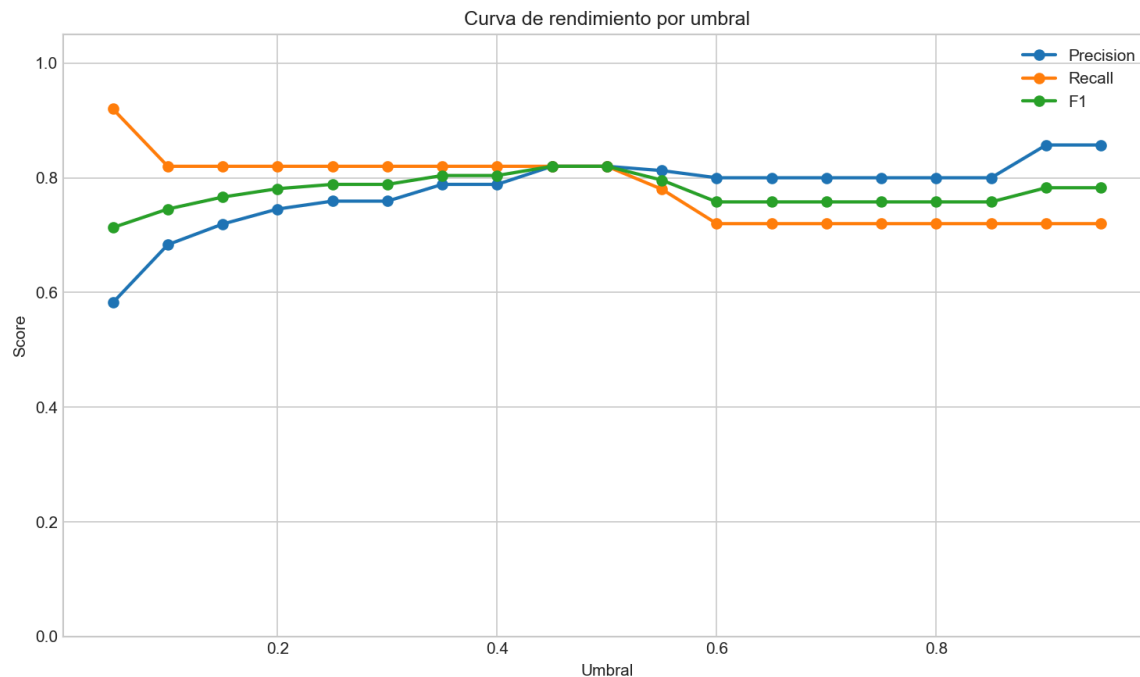


Figura 7. Análisis por umbral utilizado para comparar precisión, recall, F1 y accuracy.

5.2 Interpretación del umbral operativo

El análisis por umbral muestra que el mejor F1 de la evaluación aparece en torno a 0.45, donde precision_pos, recall_pos, f1_pos y accuracy se estabilizan en torno a 0.82. Sin embargo, el sistema opera con THRESHOLD = 0.05 para priorizar el recall y reducir la probabilidad de dejar pasar mensajes potencialmente peligrosos.

En otras palabras, con 0.05 se detectan 46 de 50 amenazas reales. A cambio, aparecen 33 falsos positivos en la clase negativa. Esa decisión es aceptable si el sistema se usa como primera criba para revisiones posteriores y se prioriza sensibilidad frente a precisión.

5.3 Matriz de confusión y distribución

La matriz de confusión del último run_id fue:

- verdaderos negativos: 17
- falsos positivos: 33

- falsos negativos: 4
- verdaderos positivos: 46

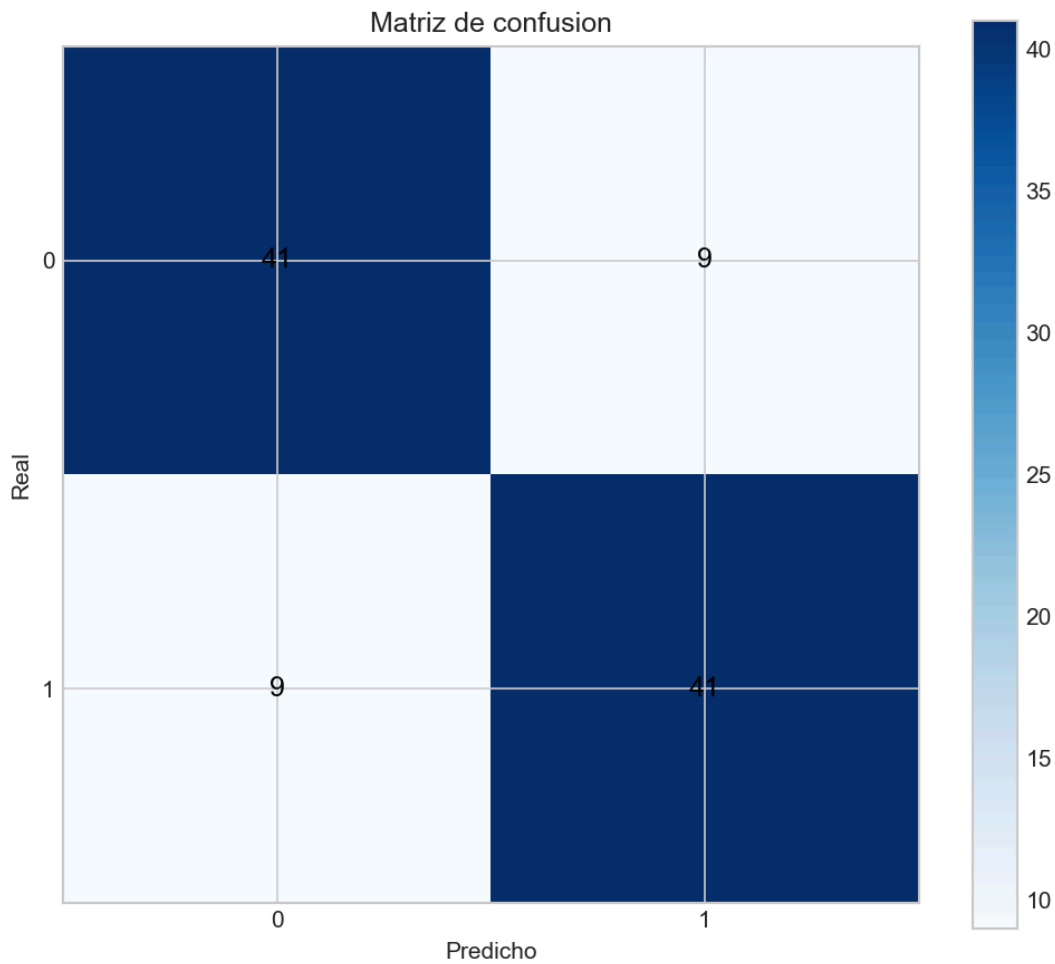


Figura 8. Matriz de confusión correspondiente a la evaluación offline del sistema.

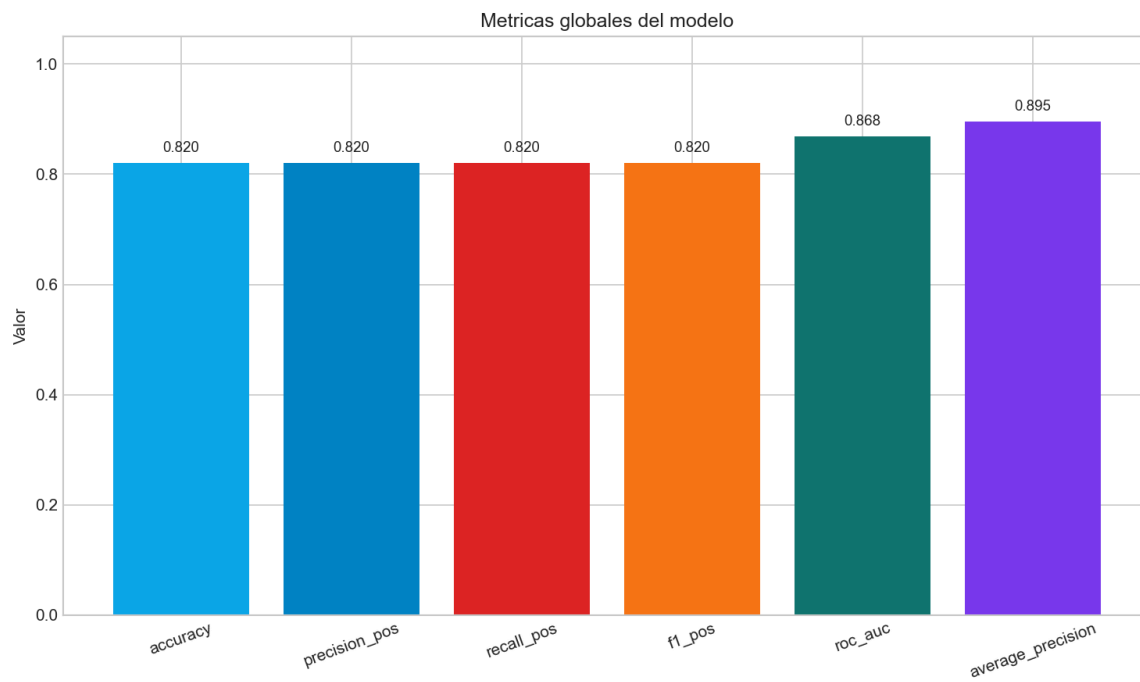


Figura 9. Métricas globales obtenidas en la evaluación offline del modelo.

Distribucion de clases del dataset de evaluacion

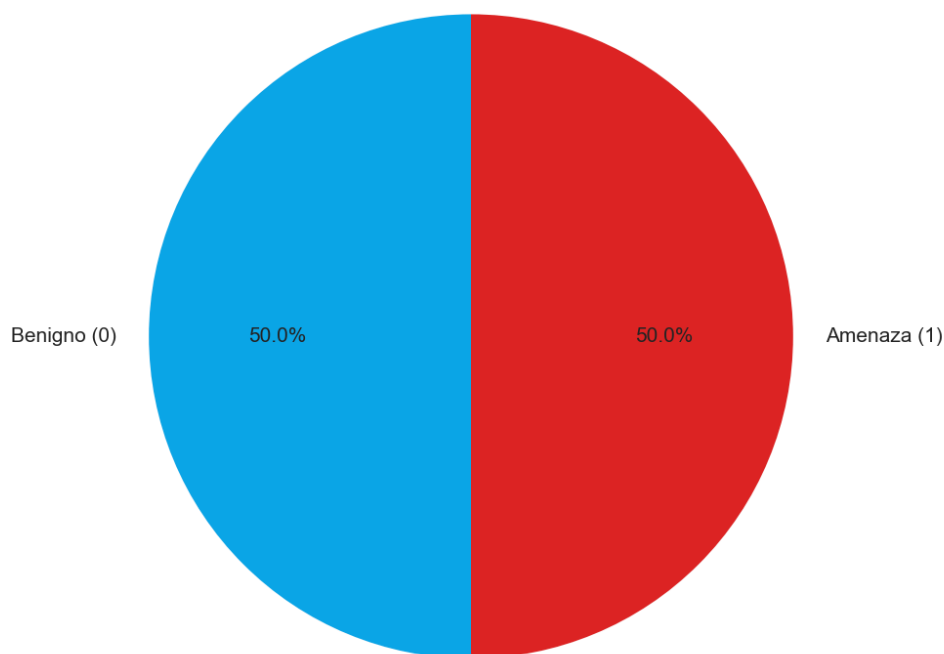


Figura 10. Distribución de clases utilizada en la evaluación offline.

El comportamiento buscado es un detector sensible. El sistema recupera muchas amenazas, pero genera un volumen apreciable de revisiones adicionales. Desde el punto de vista del pipeline, esto refuerza la necesidad de conservar trazabilidad y filtros posteriores para que la carga humana sea asumible.

5.4 Evidencia de validación integrada

La validación cuantitativa ya no se limita a confirmar que existen artefactos. El benchmark controlado más reciente, almacenado en `reports/benchmarks/bench-20260402T153415Z-b54ed0d2/benchmark_summary.json`, ejecuta escenarios de 5 segundos por carga objetivo y permite contrastar throughput, latencia y recursos del proceso sobre la misma base técnica del worker.

- Escenario 1 msg/s: throughput real 1.24 msg/s, latencia media 171.71 ms, p95 513.19 ms y tasa de error 0%.
- Escenario 5 msg/s: throughput real 5.16 msg/s, latencia media 50.04 ms, p95 57.95 ms y tasa de error 0%.
- Escenario 10 msg/s: throughput real 10.09 msg/s, latencia media 52.29 ms, p95 63.99 ms y tasa de error 0%.
- Escenario 20 msg/s: throughput real 19.08 msg/s, latencia media 52.4 ms, p95 66.06 ms y tasa de error 0%.

El escenario más exigente, ejecutado sin escritura en MongoDB para aislar el coste de inferencia y orquestación, sostuvo 19.08 msg/s con latencia media 52.4 ms, p95 66.06 ms, uso medio de CPU del 773.57% del proceso y RSS medio de 0.88 GiB. Junto a ello, el script `scripts/run_phase5_checks.py` y la CI del repositorio verifican la presencia de artefactos y el health básico del sistema.

5.5 Limitaciones detectadas

Durante la evaluación se han identificado varias limitaciones:

- El umbral operativo bajo mejora recall, pero genera un número alto de falsos positivos.
- El benchmark se ha ejecutado en un único nodo local y, en su forma de referencia, sin escritura real en MongoDB, por lo que no sustituye a una prueba distribuida completa.
- RabbitMQ desacopla productor y worker, pero el proyecto todavía no incorpora políticas automáticas de reintento, ni escalado horizontal gestionado.
- Prometheus aporta observabilidad real, aunque los dashboards y alertas siguen teniendo alcance académico y no de operación real.
- El sistema actual clasifica de forma binaria y mantiene un preprocesado deliberadamente conservador; podría enriquecerse con más contexto, explicabilidad o categorías de amenaza.

6. Conclusiones y líneas futuras

El trabajo desarrollado cumple su objetivo principal de construir un pipeline funcional y defendible para detectar mensajes de phishing o actividad sospechosa en Telegram desde una perspectiva propia de la Ingeniería de Computadores. La arquitectura producer -> RabbitMQ -> worker -> MongoDB -> API -> React, reforzada con Prometheus y Grafana, permite explicar con claridad cómo se integra cada pieza y que evidencia técnica deja a su paso.

El proyecto demuestra que es posible combinar:

- Captura automatizada de mensajes y publicación en cola
- Inferencia con un modelo de lenguaje y latencias por etapa
- Almacenamiento trazable y minimizado
- API reutilizable y métricas Prometheus
- Visualización operativa y benchmark controlado
- Validación integrada por artefactos y pruebas

La memoria permite justificar que la IA integrada responde a un proceso previo de entrenamiento documentado y no a la selección casual de un modelo externo. Sin embargo, el foco académico del trabajo no se desplaza hacia la búsqueda del mejor clasificador, sino hacia la integración de este clasificador dentro de un sistema medible y operable.

Desde el punto de vista cuantitativo, la tasa de error de prueba del 37% confirma que el sistema no debe interpretarse como un clasificador final automático, más bien como el primer cribado dentro del proceso de detección de phishing . Al mismo tiempo, el benchmark muestra que el pipeline puede sostener en local aproximadamente 19.08 mensajes por segundo con latencias medias cercanas a 52.4 ms, lo que refuerza su viabilidad como prototipo de sistemas.

Cómo miradas hacia un trabajo futuro, resultaría razonable:

- Explorar umbrales adaptativos o políticas por contexto
- Ampliar el worker con consumidores paralelos, reintentos y colas de error
- Enriquecer la observabilidad con alertas, SLOs y benchmark con escritura real en MongoDB
- Incorporar mecanismos más ricos de explicabilidad y clasificación multiclase

En su estado actual, el peso está más en la arquitectura del pipeline, la integración de componentes, la observabilidad y la evaluación del sistema como infraestructura operativa.

Bibliografía

- Akinyele, A. R., Ajayi, O. O., Munyaneza, G., Ibecheozor, U. H., & Gopakumar, N. (2024). Leveraging Generative Artificial Intelligence for cybersecurity: Analyzing diffusion models in detecting and mitigating cyber threats. *GSC Advanced Research and Reviews*, 21(2), 1-14.
- Chatzimarkaki, G., Karagiorgou, S., Konidi, M., Alexandrou, D., Bouras, T., & Evangelatos, S. (2023). Harvesting large textual and multimedia data to detect illegal activities on dark web marketplaces. *IEEE International Conference on Big Data*, 4046-4051.
- Duggineni, S. (2024). AI and machine learning applications in industrial automation and cybersecurity. *Journal of Artificial Intelligence & Cloud Computing*, 2(4), 1-5.
- Elbes, M., Hendawi, S., AlZu'bi, S., Kanan, T., & Mughaid, A. (2023). Unleashing the full potential of artificial intelligence and machine learning in cybersecurity vulnerability management. *International Conference on Information Technology*, 276-283.
- Europol. (2022). Internet Organised Crime Threat Assessment (IOCTA) 2022. European Union Agency for Law Enforcement Cooperation.
- Gartner. (2023). Top Strategic Technology Trends for 2023. Gartner, Inc.
- Guo, Y., Wang, D., Wang, L., Fang, Y., Wang, C., Yang, M., Liu, T., & Wang, H. (2024). Beyond App Markets: Demystifying underground mobile app distribution via Telegram. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(3), 1-25.
- Hummelholm, A., Iturbe, E., Krichen, M., et al. (2023). Artificial intelligence and cyber threat monitoring: Emerging approaches for automated detection. *Journal of Cybersecurity Engineering*, 5(2), 45-63.
- INCIBE. (2024). Buenas prácticas de ciberseguridad para la detección y gestión de phishing. Instituto Nacional de Ciberseguridad.
- Roy, S., et al. (2024). Characterizing cybercriminal ecosystems on Telegram at scale. *Security and Privacy Workshops*, 1-12.